Đề thi Java: Lý thuyết (chuẩn OCP) và Thực hành nghiệp vụ

Oct 5, 2026 · @Phạm Ngọc Sơn

Thông tin chung

Bộ đề gồm 2 đề độc lập, dùng Java 17 (LTS) làm chuẩn: Đề 1 kiểm tra kiến thức ngôn ngữ theo khung OCP Java SE 17 Developer (1Z0-829), Đề 2 kiểm tra khả năng hiện thực nghiệp vụ tài chính và bán hàng bằng Java thuần.

Hạng mục	Đề 1 – Lý thuyết	Đề 2 – Thực hành
Hình thức	Trắc nghiệm đọc code, 1 hoặc nhiều đáp án	Lập trình trên máy, nộp Git repo
Số câu / bài	50 câu	5 bài (3 trung bình, 2 khó)
Thời gian	90 phút	240 phút
Thang điểm	1 điểm/câu, không trừ điểm sai	100 điểm (15 + 15 + 15 + 25 + 30)
Ngưỡng đạt	34/50 (68%, như OCP)	60/100 và không bài nào dưới 30% số điểm của bài
Tài liệu	Không được dùng IDE, tài liệu	Được dùng IDE và Javadoc; không dùng công cụ AI sinh code

Quy ước chung cho cả hai đề: mọi đoạn code giả định đã có đủ `import` cần thiết và nằm trong một phương thức `main` hợp lệ, trừ khi câu hỏi nói khác. Câu có nhiều đáp án ghi rõ “Chọn 2”; phải chọn đúng đủ mới được điểm.

Đề 1 – Lý thuyết (cấu trúc OCP 1Z0-829)

50 câu phân bổ theo 11 nhóm mục tiêu của kỳ thi OCP Java SE 17; trọng số lớn nhất rơi vào OOP và Stream/Lambda.

#	Nhóm mục tiêu OCP	Câu	Số câu
1	Kiểu dữ liệu, String, ngày giờ, số	1–7	7

#	Nhóm mục tiêu OCP	Câu	Số câu
2	Điều khiển luồng	8–11	4
3	Lập trình hướng đối tượng (class, record, enum, sealed, interface)	12–21	10
4	Xử lý ngoại lệ	22–26	5
5	Mảng, Collections, Generics	27–31	5
6	Stream API và Lambda	32–39	8
7	Đóng gói, Module	40–41	2
8	Đồng thời (Concurrency)	42–45	4
9	I/O và NIO.2	46–47	2
10	JDBC	48–49	2
11	Bản địa hóa (Localization)	50	1

Nhóm 1 – Kiểu dữ liệu, String, ngày giờ

Câu 1. Kết quả in ra là gì?

```
var s = "Java";
s.concat(" 17");
s = s.toUpperCase();
System.out.println(s + s.length());
```

A. `JAVA 177` B. `JAVA4` C. `Java4` D. Lỗi biên dịch

Câu 2. Kết quả in ra là gì? (mỗi dòng trong text block thụt 4 khoảng trắng, dòng “World” thụt 5)

```
String t = """
    Hello\
     World
    """;
System.out.print(t.length());
```

A. 11 B. 12 C. 13 D. Lỗi biên dịch

Câu 3. Kết quả in ra là gì?

```
Integer a = 127, b = 127, c = 128, d = 128;
System.out.println((a == b) + " " + (c == d) + " " + c.equals(d));
```

A. `true true true` B. `true false true` C. `false false true` D. `true false false`

Câu 4. Phát biểu nào đúng?

```
short x = 10;
x = x + 5;   // dòng 1
x += 5;      // dòng 2
```

A. Cả hai dòng biên dịch được B. Chỉ dòng 1 lỗi biên dịch C. Chỉ dòng 2 lỗi biên dịch D. Cả hai dòng lỗi biên dịch

Câu 5. Kết quả in ra là gì?

```
LocalDate d = LocalDate.of(2024, 1, 31);
d.plusMonths(1);
System.out.println(d.plusMonths(1).plusDays(1));
```

A. `2024-03-01` B. `2024-03-02` C. `2024-02-29` D. Ném `DateTimeException`

Câu 6. Kết quả in ra là gì?

```
var p = Period.ofYears(1).ofMonths(2);
System.out.println(p);
```

A. `P1Y2M` B. `P2M` C. `P1Y` D. Lỗi biên dịch

Câu 7. Kết quả in ra là gì?

```
var a = new BigDecimal("1.10");
var b = new BigDecimal("1.1");
System.out.println(a.equals(b) + " " + (a.compareTo(b) == 0) + " " +
a.add(b));
```

A. `true true 2.2` B. `false true 2.20` C. `false false 2.20` D. `true true 2.20`

Nhóm 2 – Điều khiển luồng

Câu 8. Kết quả in ra là gì?

```
int day = 6;
String type = switch (day) {
    case 1, 2, 3, 4, 5 -> "WEEKDAY";
    case 6, 7 -> {
        String t = "WEEKEND";
        yield t.toLowerCase();
    }
    default -> "INVALID";
};
System.out.println(type);
```

A. `WEEKEND` B. `weekend` C. `INVALID` D. Lỗi biên dịch do `yield`

Câu 9. Kết quả in ra là gì?

```
int count = 0;
outer:
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        if (j == 1) continue outer;
        if (i == 2) break outer;
        count++;
    }
}
System.out.println(count);
```

A. 1 B. 2 C. 3 D. 6

Câu 10. Kết quả in ra là gì?

```
Object o = 42;
if (o instanceof Integer i && i > 40) {
    System.out.print("A");
}
if (!(o instanceof String s)) {
    System.out.print("B");
} else {
    System.out.print(s.length());
}
```

A. `A` B. `AB` C. `B` D. Lỗi biên dịch vì `s` nằm ngoài phạm vi

Câu 11. Kết quả in ra là gì?

```
int x = 2;
switch (x) {
    case 1: System.out.print("1");
    case 2: System.out.print("2");
    case 3: System.out.print("3"); break;
    default: System.out.print("D");
}
```

A. `2` B. `23` C. `23D` D. `123`

Nhóm 3 – Lập trình hướng đối tượng

Câu 12. Kết quả in ra là gì?

```
public record Money(String currency, long amount) {
    public Money {
        if (amount < 0) throw new IllegalArgumentException();
        currency = currency.toUpperCase();
    }
}
// trong main:
var m = new Money("vnd", 1000);
System.out.println(m.currency() + m);
```

A. `vndMoney[currency=vnd, amount=1000]` B. `VNDMoney[currency=VND, amount=1000]` C. `VNDMoney@1b6d3586` D. Lỗi biên dịch: không được gán lại tham số trong compact constructor

Câu 13. Phát biểu nào đúng về `record`? (Chọn 2)

A. Record có thể kế thừa một class khác B. Record có thể implement interface C. Các thành phần của record ngầm định là `private final` D. Record có thể khai báo thêm field instance trong thân E. Record ngầm định là `abstract`

Câu 14. Dòng nào gây lỗi biên dịch? (các kiểu cùng một file)

```
sealed interface Payment permits Cash, Card {}      // dòng 1
final class Cash implements Payment {}              // dòng 2
non-sealed class Card implements Payment {}         // dòng 3
class CreditCard extends Card {}                    // dòng 4
class Voucher implements Payment {}                 // dòng 5
```

A. Dòng 3 B. Dòng 4 C. Dòng 5 D. Không dòng nào

Câu 15. Kết quả in ra là gì?

```
class Account { String type = "BASE";   String getType() { return type; } }
class Saving extends Account { String type = "SAVING"; String getType() {
return type; } }
// trong main:
Account a = new Saving();
System.out.println(a.type + " " + a.getType());
```

A. `BASE BASE` B. `SAVING SAVING` C. `BASE SAVING` D. `SAVING BASE`

Câu 16. Kết quả in ra là gì?

```
class A {
    static { System.out.print("1"); }
    { System.out.print("2"); }
    A() { System.out.print("3"); }
}
class B extends A {
    static { System.out.print("4"); }
    { System.out.print("5"); }
    B() { System.out.print("6"); }
}
// trong main:
new B(); new B();
```

A. `1423562356` B. `1423561423 56` C. `1245361245 36` D. `1425362536`

Câu 17. Kết quả in ra là gì?

```
interface Discount {
    double rate();
    default double apply(double price) { return price * (1 - rate()); }
    private void log() { }
    static Discount of(double r) { return () -> r; }
}
// trong main:
System.out.println(Discount.of(0.25).apply(200));
```

A. `150.0` B. `50.0` C. Lỗi biên dịch: interface không được có phương thức `private` D. Lỗi biên dịch: `Discount` không phải functional interface

Câu 18. Kết quả in ra là gì?

```
static void pay(long x)    { System.out.print("long "); }
static void pay(Integer x) { System.out.print("Integer "); }
static void pay(int... x)  { System.out.print("varargs "); }
// trong main:
pay(5);
```

A. `long` B. `Integer` C. `varargs` D. Lỗi biên dịch do mơ hồ

Câu 19. Kết quả in ra là gì?

```
abstract class Product {
    abstract double price();
    Product() { System.out.print(price()); }
}
class Book extends Product {
    double p = 100;
    double price() { return p; }
}
// trong main:
new Book();
```

A. `100.0` B. `0.0` C. Ném `NullPointerException` D. Lỗi biên dịch

Câu 20. Kết quả in ra là gì?

```
enum Tier {
    SILVER(0.02), GOLD(0.05), PLATINUM(0.10);
    private final double rate;
    Tier(double r) { rate = r; }
    double rate() { return rate; }
}
// trong main:
System.out.println(Tier.valueOf("GOLD").ordinal() + Tier.GOLD.rate());
```

A. `1.05` B. `2.05` C. `10.05` D. `GOLD0.05`

Câu 21. Giả định `hashCode()` mặc định của hai đối tượng khác nhau. Kết quả in ra là gì?

```
class Sku {
    final String code;
    Sku(String c) { code = c; }
    @Override public boolean equals(Object o) {
        return o instanceof Sku s && s.code.equals(code);
    }
}
// trong main:
Set<Sku> set = new HashSet<>();
set.add(new Sku("A"));
set.add(new Sku("A"));
System.out.println(set.size());
```

A. 1 B. 2 C. Lỗi biên dịch D. Ném ngoại lệ khi chạy

Nhóm 4 – Xử lý ngoại lệ

Câu 22. Kết quả in ra là gì?

```
class Res implements AutoCloseable {
    final String n;
    Res(String n) { this.n = n; }
    public void close() { System.out.print("C" + n); }
}
// trong main:
try (var a = new Res("1"); var b = new Res("2")) {
    System.out.print("T");
    throw new RuntimeException();
} catch (Exception e) {
    System.out.print("E");
} finally {
    System.out.print("F");
}
```

A. `TEC1C2F` B. `TC1C2EF` C. `TC2C1EF` D. `TEFC2C1`

Câu 23. Kiểu nào điền vào chỗ trống để code biên dịch được? (Chọn 2)

```
class Repo { void save() throws IOException { } }
class SqlRepo extends Repo {
    @Override void save() throws ______ { }
}
```

A. `Exception` B. `FileNotFoundException` C. `SQLException` D. `IllegalStateException` E. `Throwable`

Câu 24. `calc()` trả về gì?

```
static int calc() {
    int x = 1;
    try { return x; }
    finally { x = 2; }
}
```

A. 1 B. 2 C. Lỗi biên dịch D. Không xác định

Câu 25. Biết `repo.call()` khai báo `throws IOException, SQLException`. Khối nào biên dịch được khi đặt sau `try { ... }` phù hợp?

A. `catch (FileNotFoundException | IOException e) { }` với try gọi `repo.call()` và có thêm `catch (SQLException e) { }` B. `catch (IOException | SQLException e) { e = new IOException(); }` với try gọi `repo.call()` C. `catch (IOException e) { } catch (FileNotFoundException e) { } catch (SQLException e) { }` với try gọi `repo.call()` D. `catch (IllegalArgumentException | IllegalStateException e) { }` với try chỉ gọi `Integer.parseInt("1")`

Câu 26. Kết quả in ra là gì?

```
class R implements AutoCloseable {
    public void close() throws Exception { throw new Exception("close"); }
}
// trong main:
try (R r = new R()) {
    throw new IllegalStateException("body");
} catch (Exception e) {
    System.out.println(e.getMessage() + " " + e.getSuppressed().length);
}
```

A. `close 1` B. `body 0` C. `body 1` D. `close 0`

Nhóm 5 – Mảng, Collections, Generics

Câu 27. Kết quả khi chạy là gì?

```
List<String> a = List.of("A", "B");
List<String> b = new ArrayList<>(a);
b.add("C");
List<String> c = Arrays.asList("X", "Y");
c.set(0, "Z");
System.out.println(b + " " + c);
a.add("D");
```

A. In `[A, B, C] [Z, Y]` rồi kết thúc bình thường B. In `[A, B, C] [Z, Y]` rồi ném `UnsupportedOperationException` C. Ném `UnsupportedOperationException` tại `c.set(...)` D. Lỗi biên dịch

Câu 28. Bảng hạng thành viên theo doanh số tích lũy. Kết quả in ra là gì?

```
TreeMap<Integer, String> tiers = new TreeMap<>(
    Map.of(0, "BRONZE", 5_000_000, "SILVER", 20_000_000, "GOLD"));
System.out.println(tiers.floorEntry(12_000_000).getValue()
    + " " + tiers.ceilingKey(20_000_001));
```

A. `GOLD null` B. `SILVER null` C. `SILVER 20000000` D. Ném `NullPointerException`

Câu 29. Kết quả in ra là gì?

```
int[] a = {1, 3, 5, 7};
System.out.println(Arrays.binarySearch(a, 4));
```

A. -1 B. -2 C. -3 D. 2

Câu 30. Kết quả in ra là gì?

```
Deque<String> d = new ArrayDeque<>();
d.push("A");
d.offer("B");
d.addFirst("C");
System.out.println(d.pop() + d.pollLast() + d.peek());
```

A. `CBA` B. `CAB` C. `ABC` D. `BCA`

Câu 31. Những dòng nào KHÔNG biên dịch? (Chọn 2)

```
List<? extends Number> nums = new ArrayList<Integer>();
nums.add(1);                          // L1
Number n = nums.isEmpty() ? null : nums.get(0); // L2
List<? super Integer> ints = new ArrayList<Number>();
ints.add(1);                          // L3
Integer i = ints.get(0);              // L4
```

A. L1 B. L2 C. L3 D. L4

Nhóm 6 – Stream API và Lambda

Câu 32. Kết quả in ra là gì?

```
var orders = List.of(120, 80, 200, 50);
int total = orders.stream()
    .filter(x -> x >= 80)
    .mapToInt(Integer::intValue)
    .sum();
System.out.println(total);
```

A. 320 B. 400 C. 450 D. 200

Câu 33. Kết quả in ra là gì?

```
Stream.of("a", "b", "c")
      .peek(System.out::print)
      .filter(s -> s.equals("b"))
      .findFirst();
```

A. Không in gì B. `ab` C. `abc` D. `b`

Câu 34. Kết quả in ra là gì?

```
record Sale(String region, int amount) { }
var sales = List.of(new Sale("HN", 100), new Sale("HCM", 200), new Sale("HN",
50));
Map<String, Integer> m = sales.stream().collect(
    Collectors.groupingBy(Sale::region, TreeMap::new,
                          Collectors.summingInt(Sale::amount)));
System.out.println(m);
```

A. `{HN=150, HCM=200}` B. `{HCM=200, HN=150}` C. `{HCM=[200], HN=[100, 50]}` D. Lỗi biên dịch

Câu 35. Kết quả khi chạy là gì?

```
var m = Stream.of("apple", "avocado", "banana")
    .collect(Collectors.toMap(s -> s.charAt(0), String::length));
System.out.println(m);
```

A. `{a=5, b=6}` B. `{a=7, b=6}` C. `{a=12, b=6}` D. Ném `IllegalStateException`

Câu 36. Dòng nào KHÔNG biên dịch?

A. `BiFunction<Integer, Integer, Integer> f = Integer::sum;` B. `ToIntFunction<String> g = String::length;` C. `Consumer<String> c = String::length;` D. `Supplier<String> s = String::new;` E. `UnaryOperator<Integer> u = (int x) -> x + 1;`

Câu 37. Kết quả in ra là gì?

```
Optional<String> o = Optional.empty();
System.out.println(o.map(String::length).orElseGet(() -> -1)
    + " " + Optional.ofNullable(null).isPresent());
```

A. `-1 false` B. `0 false` C. Ném `NoSuchElementException` D. Ném `NullPointerException`

Câu 38. Kết quả in ra là gì?

```
var prices = List.of(new BigDecimal("10.5"), new BigDecimal("4.50"));
System.out.println(prices.stream().reduce(BigDecimal.ZERO, BigDecimal::add));
```

A. `15` B. `15.0` C. `15.00` D. `15.000`

Câu 39. Kết quả in ra là gì?

```
var names = List.of("Dung", "An", "Linh", "Bao");
names.stream()
     .sorted(Comparator.comparing(String::length)
                       .thenComparing(Comparator.reverseOrder()))
     .forEach(System.out::print);
```

A. `AnBaoDungLinh` B. `AnBaoLinhDung` C. `LinhDungBaoAn` D. Lỗi biên dịch

Nhóm 7 – Đóng gói, Module

Câu 40. Cho khai báo module sau. Phát biểu nào đúng?

```
module com.shop.order {
    requires com.shop.payment;
    exports com.shop.order.api;
}
```

A. Module khác chỉ truy cập được kiểu public trong `com.shop.order.api` B. Module nào `requires com.shop.order` cũng tự động đọc được `com.shop.payment` C. Reflection từ module khác truy cập được thành viên private trong `com.shop.order.api` D. Thiếu `requires java.base` nên lỗi biên dịch

Câu 41. Lệnh nào in ra khai báo module (requires, exports) có trong file `order.jar`?

A. `java --list-modules order.jar` B. `jar --file order.jar --describe-module` C. `javac --show-module order.jar` D. `jlink --describe order.jar`

Nhóm 8 – Đồng thời

Câu 42. Kết quả in ra là gì?

```
var counter = new AtomicInteger();
var ex = Executors.newFixedThreadPool(4);
for (int i = 0; i < 1000; i++) ex.submit(counter::incrementAndGet);
ex.shutdown();
ex.awaitTermination(1, TimeUnit.MINUTES);
System.out.println(counter.get());
```

A. Luôn là 1000 B. Một giá trị bất kỳ ⩽ 1000 C. 0 D. Ném `RejectedExecutionException`

Câu 43. Nhiều luồng cùng cộng doanh thu theo mã cửa hàng vào một map. Cách nào luôn cho tổng đúng?

A. `HashMap` với `map.put(k, map.get(k) + v)` B. `ConcurrentHashMap` với `map.put(k, map.get(k) + v)` C. `ConcurrentHashMap` với `map.merge(k, v, Integer::sum)` D. `Collections.synchronizedMap(new HashMap<>())` với `map.put(k, map.get(k) + v)` không đồng bộ thêm

Câu 44. Hàm `transfer(from, to, amt)` khóa `synchronized(from)` rồi `synchronized(to)`. Hai luồng chạy đồng thời A→B và B→A. Cách nào loại bỏ deadlock mà vẫn đảm bảo tính đúng?

A. Đánh dấu field số dư là `volatile` và bỏ khóa B. Luôn khóa hai tài khoản theo thứ tự cố định, ví dụ id tăng dần C. Gọi `Thread.yield()` giữa hai lần khóa D. Tăng số luồng trong thread pool

Câu 45. Kết quả khi chạy là gì?

```
List<Integer> out = new ArrayList<>();
IntStream.range(0, 1000).parallel().forEach(out::add);
System.out.println(out.size());
```

A. Luôn in 1000 B. Luôn in 1000, thứ tự 0..999 C. Không xác định: có thể in số nhỏ hơn 1000 hoặc ném ngoại lệ D. Lỗi biên dịch

Nhóm 9 – I/O và NIO.2

Câu 46. Kết quả in ra là gì (hệ thống Linux)?

```
Path p = Path.of("/data/reports/../invoices/./2026.csv");
Path n = p.normalize();
System.out.println(n.getFileName() + " " + n.getNameCount());
```

A. `2026.csv 3` B. `2026.csv 4` C. `/data/invoices/2026.csv 3` D. `2026.csv 6`

Câu 47. Phát biểu nào đúng?

A. `Files.readAllLines` trả về `Stream<String>` đọc lười B. `Files.lines` trả về `Stream<String>` đọc lười và cần đóng bằng try-with-resources C. `Files.writeString(path, s)` mặc định ghi nối vào cuối file D. `Files.copy(src, dst)` mặc định ghi đè nếu `dst` đã tồn tại

Nhóm 10 – JDBC

Câu 48. Phát biểu nào đúng về đoạn code trừ tiền sau?

```
String sql = "UPDATE account SET balance = balance - ? WHERE id = ? AND
balance >= ?";
try (PreparedStatement ps = conn.prepareStatement(sql)) {
    ps.setBigDecimal(1, amount);
    ps.setLong(2, accountId);
    ps.setBigDecimal(3, amount);
    int rows = ps.executeUpdate();
}
```

A. Chỉ số tham số phải bắt đầu từ 0 B. `rows == 0` nghĩa là không có dòng nào thỏa điều kiện (sai id hoặc không đủ số dư) C. Phải dùng `executeQuery` vì câu lệnh có điều kiện `WHERE` D. Phải đóng `ResultSet` trả về từ `executeUpdate`

Câu 49. Nếu `credit(...)` ném `SQLException`, điều gì xảy ra với thao tác `debit(...)`?

```
conn.setAutoCommit(false);
try {
    debit(conn, from, amount);
    credit(conn, to, amount);
    conn.commit();
} catch (SQLException e) {
    conn.rollback();
}
```

A. Đã được commit ngay sau khi `debit` chạy xong B. Bị hoàn tác bởi `rollback()` C. Được commit khi đóng kết nối D. Không xác định vì thiếu `Savepoint`

Nhóm 11 – Bản địa hóa

Câu 50. Classpath có `Messages.properties`, `Messages_en.properties`, `Messages_vi.properties`. Locale mặc định là `en_US`. Khóa `greeting` chỉ có trong `Messages_en.properties` và `Messages.properties`. Giá trị được lấy từ đâu?

```
var rb = ResourceBundle.getBundle("Messages", Locale.forLanguageTag("vi-
VN"));
System.out.println(rb.getString("greeting"));
```

A. `Messages_vi.properties` B. `Messages_en.properties` C. `Messages.properties` D. Ném `MissingResourceException`

Đề 2 – Thực hành: quy định chung và 3 bài trung bình

Học viên làm 5 bài trong 240 phút bằng Java 17 thuần, nộp một Git repo dựng bằng Maven hoặc Gradle.

Quy định kỹ thuật áp dụng cho mọi bài

- Chỉ dùng JDK 17, JUnit 5 và AssertJ; không dùng Spring, Lombok hay thư viện nghiệp vụ khác.
- Tiền tệ dùng `BigDecimal`, đơn vị VND, làm tròn đến đồng bằng `RoundingMode.HALF_UP` tại đúng điểm đề bài quy định; dùng `double` cho tiền bị trừ 50% điểm của bài.
- Mỗi bài một package `bai1` đến `bai5`, có test JUnit cho tối thiểu các ví dụ trong đề; `mvn test` (hoặc `gradle test`) phải chạy xanh.
- Dữ liệu không hợp lệ phải ném ngoại lệ nghiệp vụ có tên rõ ràng, không trả về `null` hay in lỗi ra console.
- Ưu tiên `record`, `enum`, `sealed interface`, Stream API và đối tượng bất biến khi phù hợp.
- File `README.md` nêu giả định, quyết định thiết kế và phần chưa hoàn thành.

Bài 1 – Tính hóa đơn bán lẻ (Trung bình, 15 điểm)

Bối cảnh. Chuỗi siêu thị cần module tính tiền cho một hóa đơn gồm nhiều dòng hàng, mỗi nhóm hàng chịu thuế suất GTGT khác nhau.

Quy tắc nghiệp vụ.

1. Tiền hàng dòng = đơn giá × số lượng; chiết khấu dòng = tiền hàng × % chiết khấu dòng (0–50%), làm tròn đến đồng.
2. Nếu tổng tiền sau chiết khấu dòng ≥ 2.000.000 đ thì giảm thêm 2% trên toàn hóa đơn.
3. Khoản giảm hóa đơn được phân bổ về từng dòng theo tỷ lệ giá trị sau chiết khấu dòng, làm tròn đến đồng; phần lệch do làm tròn dồn vào dòng có giá trị lớn nhất.
4. Thuế GTGT tính theo từng dòng trên giá trị sau mọi khoản giảm, làm tròn đến đồng. Thuế suất cấu hình theo nhóm: `THUC_PHAM` 5%, `GIA_DUNG` 10%, `DIEN_TU` 8%, `SACH` 0%.
5. Từ chối hóa đơn rỗng, số lượng ≤ 0, đơn giá âm, chiết khấu ngoài khoảng, hoặc trùng mã sản phẩm trên hai dòng.

Yêu cầu. Viết `InvoiceCalculator.calculate(List<InvoiceLine>)` trả về `InvoiceResult` gồm chi tiết từng dòng (tiền hàng, chiết khấu dòng, giảm phân bổ, giá trị tính thuế, thuế) và tổng hóa đơn; có hàm tổng hợp thuế theo từng mức thuế suất.

Ví dụ kiểm tra.

Mã	Nhóm	Đơn giá (đ)	SL	CK dòng
SP01	THUC_PHAM	32.000	10	5%
SP02	GIA_DUNG	1.250.000	2	0%
SP03	SACH	45.000	3	0%

Kết quả đúng:

Mã	Sau CK dòng (đ)	Giảm phân bổ (đ)	Tính thuế (đ)	Thuế (đ)
SP01	304.000	6.080	297.920	14.896
SP02	2.500.000	50.000	2.450.000	245.000
SP03	135.000	2.700	132.300	0
Tổng	2.939.000	58.780	2.880.220	259.896

Tổng thanh toán: 3.140.116 đ.

Bài 2 – Lịch trả nợ khoản vay tiêu dùng (Trung bình, 15 điểm)

Bối cảnh. Công ty tài chính cần sinh lịch trả nợ hàng tháng theo hai phương pháp để khách hàng so sánh trước khi ký hợp đồng.

Quy tắc nghiệp vụ.

1. Đầu vào: số tiền vay, lãi suất năm, số kỳ (tháng), ngày giải ngân. Lãi tháng = lãi năm / 12.
2. Phương pháp `GOC_DEU`: gốc mỗi kỳ = số tiền vay / số kỳ, lãi tính trên dư nợ đầu kỳ.
3. Phương pháp `TRA_DEU` (annuity): tổng trả mỗi kỳ = P × r / (1 − (1 + r)^−n), làm tròn đến đồng; lãi = dư nợ đầu kỳ × r làm tròn đến đồng; gốc = tổng trả − lãi.
4. Kỳ cuối của cả hai phương pháp trả toàn bộ dư nợ còn lại để dư nợ cuối băng đúng 0.
5. Ngày đến hạn kỳ k = ngày giải ngân + k tháng (tính từ ngày gốc, không cộng dồn từ kỳ trước). Rơi vào Thứ Bảy hoặc Chủ nhật thì dời sang Thứ Hai kế tiếp; lãi vẫn tính trọn tháng.
6. Từ chối số tiền ⩽ 0, lãi suất âm hoặc trên 50%/năm, số kỳ ngoài 1–360.

Yêu cầu. Viết `LoanScheduler.generate(LoanRequest, Method)` trả về danh sách `Installment(k, dueDate, openingBalance, principal, interest, payment, closingBalance)` và hàm so sánh tổng lãi giữa hai phương pháp.

Ví dụ kiểm tra. Vay 120.000.000 đ, lãi 12%/năm, 6 kỳ, giải ngân 31/01/2026.

Kỳ	Ngày đến hạn	GOC_DEU: trả (đ)	TRA_DEU: lãi (đ)	TRA_DEU: gốc (đ)	TRA_DEU: dư nợ cuối (đ)
1	02/03/2026 (28/02 là Thứ Bảy)	21.200.000	1.200.000	19.505.804	100.494.196
2	31/03/2026	21.000.000	1.004.942	19.700.862	80.793.334
3	30/04/2026	20.800.000	807.933	19.897.871	60.895.463
4	01/06/2026 (31/05 là Chủ nhật)	20.600.000	608.955	20.096.849	40.798.614
5	30/06/2026	20.400.000	407.986	20.297.818	20.500.796
6	31/07/2026	20.200.000	205.008	20.500.796	0

TRA_DEU trả 20.705.804 đ mỗi kỳ. Tổng lãi: GOC_DEU 4.200.000 đ, TRA_DEU 4.234.824 đ.

Bài 3 – Báo cáo doanh số từ file CSV (Trung bình, 15 điểm)

Bối cảnh. Phòng kinh doanh xuất dữ liệu đơn hàng ra CSV và cần báo cáo doanh thu thuần theo nhiều chiều. Dữ liệu thực tế có dòng lỗi.

Quy tắc nghiệp vụ.

1. Cột: `orderId,orderDate,region,productCode,category,quantity,unitPrice,status`; dòng đầu là tiêu đề.
2. Giá trị dòng = quantity × unitPrice. `COMPLETED` ghi tăng doanh thu, `RETURNED` ghi giảm, `CANCELLED` bỏ qua.
3. Dòng lỗi (sai số cột, ngày không hợp lệ, số lượng/đơn giá không phải số dương, trạng thái lạ, `orderId` trùng dòng trước) bị loại và ghi vào danh sách lỗi kèm số dòng và lý do; chương trình không được dừng.
4. Tỷ lệ hoàn trả theo nhóm hàng = giá trị RETURNED / giá trị COMPLETED, hiển thị phần trăm 2 chữ số thập phân.

Yêu cầu. Đọc file bằng `Files.lines`, xử lý bằng Stream API (không dùng vòng lặp cho phần tổng hợp) và trả về `SalesReport` gồm: doanh thu thuần theo vùng (giảm dần), top 3 sản phẩm theo doanh thu thuần, doanh thu thuần theo tháng (`YearMonth` tăng dần), tỷ lệ hoàn trả theo nhóm hàng, danh sách lỗi.

Dữ liệu mẫu.

```
orderId,orderDate,region,productCode,category,quantity,unitPrice,status
O001,2026-07-03,HN,P01,DIEN_TU,2,5000000,COMPLETED
O002,2026-07-05,HCM,P02,GIA_DUNG,5,400000,COMPLETED
O003,2026-07-15,HN,P03,THUC_PHAM,20,35000,COMPLETED
O004,2026-07-20,DN,P01,DIEN_TU,1,5000000,RETURNED
O005,2026-08-02,HCM,P01,DIEN_TU,3,4800000,COMPLETED
O006,2026-08-10,HN,P02,GIA_DUNG,2,400000,CANCELLED
O007,2026-08-12,DN,P04,GIA_DUNG,4,750000,COMPLETED
O008,2026-08-18,HCM,P03,THUC_PHAM,abc,35000,COMPLETED
O009,2026-08-25,HN,P04,GIA_DUNG,1,750000,RETURNED
O010,2026-09-01,HCM,P02,GIA_DUNG,10,380000,COMPLETED
O011,2026-13-05,HN,P01,DIEN_TU,1,5000000,COMPLETED
O002,2026-09-03,HCM,P02,GIA_DUNG,1,400000,COMPLETED
```

Kết quả đúng.

- 3 dòng lỗi: dòng 9 (O008, số lượng không hợp lệ), dòng 12 (O011, ngày không hợp lệ), dòng 13 (O002, trùng mã đơn). Số dòng tính cả dòng tiêu đề.
- Theo vùng: HCM 20.200.000 đ; HN 9.950.000 đ; DN −2.000.000 đ. Tổng 28.150.000 đ.
- Top 3 sản phẩm: P01 19.400.000 đ; P02 5.800.000 đ; P04 2.250.000 đ.
- Theo tháng: 2026-07 7.700.000 đ; 2026-08 16.650.000 đ; 2026-09 3.800.000 đ.
- Tỷ lệ hoàn trả: DIEN_TU 20,49%; GIA_DUNG 8,52%; THUC_PHAM 0,00%.

Đề 2 – Thực hành: 2 bài khó

Hai bài khó chiếm 55 điểm, đánh giá khả năng thiết kế mô hình nghiệp vụ, xử lý đồng thời và bài toán tối ưu, không chỉ cú pháp.

Bài 4 – Sổ cái chuyển tiền nội bộ (Khó, 25 điểm)

Bối cảnh. Ví điện tử cần lõi chuyển tiền giữa các tài khoản khách hàng, ghi sổ theo nguyên tắc bút toán kép, chạy đúng khi nhiều yêu cầu đến đồng thời và khi client gửi lại cùng một yêu cầu.

Quy tắc nghiệp vụ.

1. Mỗi giao dịch sinh một bút toán (`JournalEntry`) gồm nhiều dòng Nợ/Có; tổng Nợ phải bằng tổng Có, nếu không thì từ chối ghi sổ. Số dư tài khoản chỉ thay đổi qua bút toán, không có setter.
2. Phí chuyển = 0,1% số tiền, làm tròn đến đồng, tối thiểu 2.000 đ, tối đa 50.000 đ, người gửi chịu. Bút toán gồm 3 dòng: Nợ tài khoản gửi (số tiền + phí), Có tài khoản nhận (số tiền), Có tài khoản `FEE_INCOME` (phí).

3. Số dư sau giao dịch không được thấp hơn −hạn mức thấu chi của tài khoản (mặc định 0). Tổng tiền chuyển đi của một tài khoản trong một ngày không vượt 500.000.000 đ (không tính phí).
4. Mỗi yêu cầu có `idempotencyKey`. Gửi lại cùng khóa với cùng nội dung trả về kết quả cũ, không ghi sổ lần hai; cùng khóa khác nội dung thì ném `IdempotencyConflictException`. Yêu cầu bị từ chối cũng được ghi nhớ kết quả từ chối.
5. `reverse(journalId)` tạo bút toán đảo ngược toàn bộ (kể cả phí); một bút toán chỉ đảo được một lần; bút toán đảo không áp hạn mức ngày nhưng vẫn kiểm tra thấu chi của bên bị trừ.
6. Sao kê tài khoản theo khoảng ngày: số dư đầu kỳ, từng dòng phát sinh, số dư cuối kỳ. Bảng cân đối thử: tổng số dư mọi tài khoản (kể cả `FEE_INCOME`) luôn bằng tổng số dư ban đầu.

Yêu cầu kỹ thuật.

- `LedgerService` với các thao tác `openAccount`, `transfer`, `reverse`, `statement`, `trialBalance`; thời gian lấy từ `Clock` được inject để test được hạn mức ngày.
- Thread-safe, không deadlock, không dùng một khóa toàn cục cho mọi giao dịch (hai giao dịch không chung tài khoản khách hàng phải chạy song song được).
- Test tải bắt buộc: 50 tài khoản, 8 luồng, mỗi luồng 10.000 giao dịch ngẫu nhiên (có 10% gửi lại khóa cũ); kết thúc trong 30 giây, bảng cân đối thử khớp, không tài khoản nào vượt thấu chi, số bút toán bằng số khóa duy nhất được chấp nhận.

Ví dụ kiểm tra. A có 10.000.000 đ, B có 0 đ, thấu chi 0.

Bước	Thao tác	Kết quả
1	`transfer(k1, A→B, 1.000.000)`	Phí 2.000 (0,1% = 1.000 < tối thiểu). A = 8.998.000; B = 1.000.000; FEE_INCOME = 2.000
2	`transfer(k1, A→B, 1.000.000)`	Trả kết quả bước 1, số dư không đổi
3	`transfer(k1, A→B, 2.000.000)`	`IdempotencyConflictException`
4	`transfer(k2, A→B, 9.000.000)`	Phí 9.000; cần 9.009.000 > 8.998.000 nên từ chối `InsufficientFundsException`
5	`reverse(journal bước 1)`	A = 10.000.000; B = 0; FEE_INCOME = 0
6	`reverse(journal bước 1)` lần 2	`AlreadyReversedException`

Với giao dịch 80.000.000 đ, phí tính ra 80.000 đ bị chặn trên còn 50.000 đ.

Bài 5 – Động cơ khuyến mại tối ưu cho giỏ hàng (Khó, 30 điểm)

Bối cảnh. Trang thương mại điện tử chạy nhiều chương trình khuyến mại cùng lúc. Hệ thống phải tự chọn cách áp dụng có lợi nhất cho khách và giải thích được kết quả cho bộ phận chăm sóc khách hàng.

Các loại khuyến mại (mô hình bằng `sealed interface Promotion`):

Loại	Cấp	Mô tả
`BuyXGetY`	Dòng	Mua X tặng Y cùng sản phẩm; mỗi bộ X+Y đơn vị được giảm giá của Y đơn vị
`CategoryPercent`	Dòng	Giảm p% cho từng đơn vị thuộc nhóm hàng, có trần tổng giảm tùy chọn
`Bundle`	Dòng	Combo một tập sản phẩm (mỗi loại 1 đơn vị) bán với giá cố định
`OrderThreshold`	Đơn hàng	Giá trị sau khuyến mại dòng ≥ ngưỡng thì giảm số tiền cố định
`Voucher`	Đơn hàng	Mã khách nhập; giảm p% trên giá trị sau mọi khuyến mại khác, có trần; tối đa 1 voucher/đơn

Quy tắc nghiệp vụ.

1. Mỗi khuyến mại có thời gian hiệu lực, hạng khách áp dụng và độ ưu tiên; khuyến mại hết hạn hoặc sai hạng khách bị loại trước khi tính.
2. Mỗi đơn vị hàng chỉ được thuộc tối đa một khuyến mại cấp dòng. Các đơn vị khác nhau của cùng một sản phẩm có thể thuộc các khuyến mại khác nhau hoặc không thuộc khuyến mại nào.
3. Khuyến mại cấp đơn áp theo thứ tự: `OrderThreshold` rồi `Voucher`, tự động khi đủ điều kiện.
4. Một phương án = cách gán đơn vị hàng vào khuyến mại dòng + các khuyến mại cấp đơn kéo theo. Engine chọn phương án có số tiền khách trả nhỏ nhất; hòa thì chọn phương án dùng ít khuyến mại dòng hơn; vẫn hòa thì chọn phương án có tổng độ ưu tiên cao hơn.
5. Mọi khoản giảm làm tròn đến đồng; số tiền phải trả không âm.

Yêu cầu kỹ thuật.

- `PromotionEngine.evaluate(Cart, List<Promotion>, Customer, Optional<String> voucherCode, LocalDateTime at)` trả về `PricingResult`: tổng gốc, danh sách khuyến mại được áp (loại, đơn vị bị chiếm, số tiền giảm), số tiền phải trả và một đoạn

giải thích dạng văn bản.

- Giỏ tối đa 30 dòng, 15 khuyến mại; mỗi lần tính dưới 1 giây. Duyệt vét cạn ngây thơ sẽ không đạt; README phải mô tả chiến lược tìm kiếm (ví dụ quy hoạch động theo số đơn vị, nhánh cận) và độ phức tạp.
- Thêm loại khuyến mại mới không được phải sửa logic tìm kiếm cốt lõi.

Ví dụ kiểm tra. Giỏ hàng tổng gốc 170.000 đ:

Sản phẩm	Nhóm	Đơn giá (đ)	SL
SP-A Nước ngọt	DO_UONG	10.000	6
SP-B Bánh quy	BANH_KEO	25.000	2
SP-C Dầu ăn	GIA_VI	60.000	1

Khuyến mại đang chạy: P1 `BuyXGetY` SP-A mua 2 tặng 1; P2 `CategoryPercent` DO_UONG giảm 25%; P3 `Bundle` 1 SP-A + 1 SP-B giá 30.000 đ; P4 `OrderThreshold` từ 150.000 đ giảm 15.000 đ; P5 `Voucher` SALE10 giảm 10%, tối đa 12.000 đ. Khách nhập SALE10.

Kết quả đúng: áp P1 cho 6 SP-A (2 bộ, giảm 20.000 đ), P4 giảm 15.000 đ, P5 giảm 12.000 đ (10% của 135.000 đ là 13.500 đ, bị chặn). Khách trả 123.000 đ.

Bẫy cần tránh: phương án có giảm cấp dòng lớn nhất (2 combo P3 + 1 bộ P1 + P2 cho 1 lon còn lại, giảm 22.500 đ) đưa giá trị xuống 147.500 đ, mất P4 và khách phải trả 135.500 đ. Ba phương án khác cũng giảm đúng 20.000 đ cấp dòng nhưng dùng 2–3 khuyến mại dòng nên thua theo quy tắc hòa.